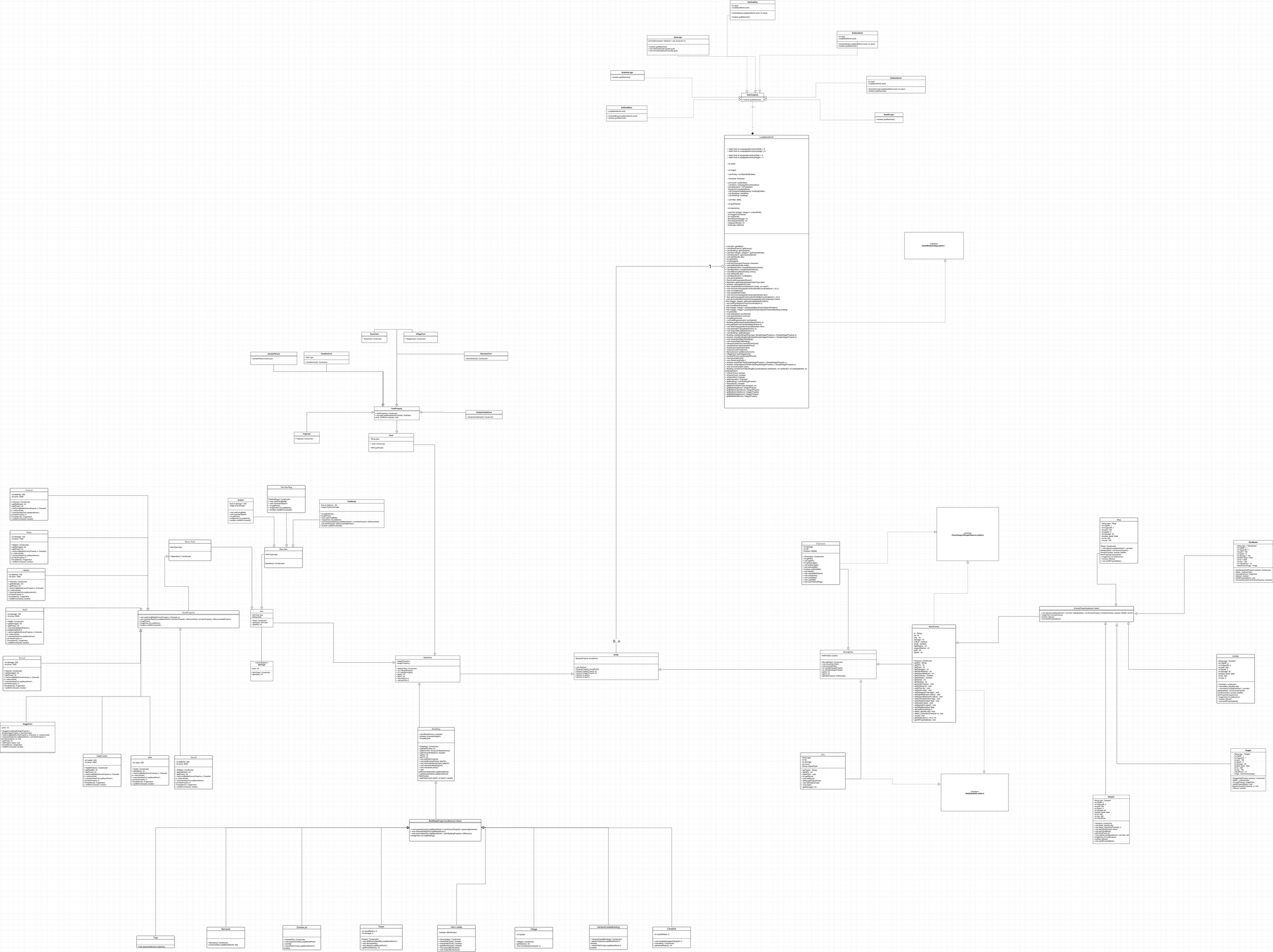
SubGoalBop
- int value;
- LoopIterator&world world;
+ SubGoalBop(LoopIterator&world world, int value);
+ boolean goalReached();
SubGoalGold
- int value;
- LoopIterator&world world;
+ SubGoalGold(LoopIterator&world world, int value);
+ boolean goalReached();
GoalLogic
- List<GoalComputer> totalGoal = new ArrayList<>();
+ boolean goalReached();
+ void addGoalGoal(GoalComputer goal);
+ void removeGoalGoal(GoalComputer goal);
GoalAteLogic
+ boolean goalReached();
GoalComputer
+ boolean goalReached();
SubGoalCycle
- int value;
- LoopIterator&world world;
+ SubGoalCycle(LoopIterator&world world, int value);
+ boolean goalReached();
SubGoalBoss
- LoopIterator&world world;
+ SubGoalBoss(LoopIterator&world world);
+ boolean goalReached();
GoalKiLogic
+ boolean goalReached();
LoopIterator&World
+ static final int unequippedInventoryWidth = 4;
+ static final int unequippedInventoryHeight = 4;
+ static final int equippedInventoryWidth = 4;
+ static final int equippedInventoryHeight = 1;
- int width;
- int height;
- List<Entity> nonSpecifiedEntities;
- Character character;
- List<Card> cardEntities;
- List<Item> unequippedInventoryItems;
- List<BaseItem> inPlayItems;
- List<Enemy> activeEnemies;
- List<TemporaryCastleBuilding> buildingEntities;
- List<Building> spawners;
- List<Building> buildings;
- List<Ally> allies;
- int goldOwned;
- int experience;
- List<Pair<Integer, Integer>> orderedPath;
- int currentPathPosition;
- int bogOwned;
- int shouldSpawnBoggle = 1;
- ShouldspawnRuble: int
- hasspawnRuble: int
- GoalLogic totalGoal
+ List<Ally> getAllies();
+ List<BaseEnemy> getEnemy();
+ List<Building> getCampfire();
+ List<Pair<Integer, Integer>> getOrderedPath();
+ List<BaseItem> getEquipedItems();
+ void addAlly(Ally ally);
+ void setWidth(int w);
+ int getHeight();
+ void setCharacter(Character character);
+ void setHeight(int h);
+ List<BaseEnemy> possibleSpawnEnemies();
+ void addEnemy(BaseEnemy enemy);
+ void killEnemy(BaseEnemy enemy);
+ int getWidth();
+ List<BaseEnemy> getEnemies();
+ void setNonSpecifiedEntities(List);
+ Character getCharacter();
+ boolean addUnequippedItem(Item);
+ boolean canEquipItem(int slot);
+ void addToInventory(BaseItem item, int noItem);
+ void removeUnequippedInventoryItem(Coordinates(int x, int y));
+ void setBogOwned(int);
+ void updatePathCircle();
+ void addUnequippedInventoryItem(Entity item);
+ Item getUnequippedInventoryItemEntityBy(Coordinates(int x, int y));
+ void removeItemByPosition(UnequippedInventoryItem(int index);
+ void setGoldOwned(int goldOwned);
+ void addEnemyOverEventCoordinates(int x);
+ void moveBeastEnemies();
+ Pair<Integer, Integer> possibleGetBeastEnemySpawnPosition();
+ Pair<Integer, Integer> possibleGetVampireSpawnPosition(Building building);
+ int getGold();
+ void spawnGold(int numGained);
+ void spawnGold(int numLost);
+ void setGold(int);
+ void addExperience(int numGained);
+ void addEnemyOverEventCoordinates(Entity e);
+ void getBeastFromCampfire(BaseEnemy e);
+ void addUnequippedInventory(BaseItem item);
+ void addEnemyOverEventCoordinates(Entity e);
+ void supportBeastBaseEnemy e);
+ List<Building> getBuildings();
+ boolean checkIfBuildingAt(SimpleIntegerProperty x, SimpleIntegerProperty y);
+ boolean checkBuildingAt(int x,SimpleIntegerProperty x, SimpleIntegerProperty y);
+ void characterStepOn(Building);
+ void characterStepOn();
+ VampireCastleCard hasVampireCard();
+ CampfireCard hasCampfireCard();
+ TowerCard hasTowerCard();
+ TrapCard hasTrapCard();
+ BarracksCard hasBarracksCard();
+ VillageCard hasVillageCard();
+ List<Card> getCardEntities();
+ Card getCard(Card);
+ void checkCardEntity();
+ boolean checkAt(SimpleIntegerProperty x, SimpleIntegerProperty y);
+ boolean checkAdjacentTo(PathTile(SimpleIntegerProperty x, SimpleIntegerProperty y);
+ boolean checkAdjacentTo(int x,int y);
+ Building convertCardToBuildingBy(Coordinates(int cardNodeX, int cardNodeY, int buildingNodeX, int buildingNodeY);
+ int getNum();
+ isDayTime(): boolean
+ isCamarDown(): boolean
+ int getCamarDown();
+ getCharacter(): Character
+ getBuildings(): List<BuildingProperty>
+ getAllies(): List<Ally>
+ getItemFromNextTypeItem(type): int
+ getBeastBogBoun(): IntegerProperty
+ getBeastCrimsonAura(): IntegerProperty
+ getBeastCrimsonAura(): IntegerProperty
+ getBeastBoneAura(): IntegerProperty
+ getBeastBeetleAura(): IntegerProperty
«interface»
GameMechanics pattern»
Character
- int damage
- int HP
- boolean inBattle
+ Character(): Constructor
+ int getHP()
+ void setHP()
+ int getDamage()
+ void setDamage()
+ void setInBattle()
+ boolean getInBattle()
+ void attack()
+ void setPathPosition()
+ void useItem()
+ void setItem()
+ void setBeft()
+ void setTheCastling()
«interface»
EnemySupportMergeObserver pattern»
Slug
- String type "Slug"
- int hp = 50
- int bugsoHP = 0
- int gold = 50
- int bugsoGold = 0
- int damage = 0
- int experience = 50 beta
- int bp = 500
- int exp = 50
Slug(): Constructor
+ void attack(LoopIterator&world, List<Ally> deltaAllies, List<EnemyProperty> deltaEnemies);
+ void setEnemyPropertyBeta(EnemyProperty bevPropertyA[]) assignment();
+ boolean getBeastBogBoun();
+ boolean isBoss();
+ void addPropertyBeta();
ElenBeaste
- String type = "ElenBeaste"
- int hp=50 = 0
- int bugsoHP = 0
- int gold = 150
- int bugsoGold = 0
- int damage = 150
- boolean weak = false
- int bp = 500
- int exp = 150
- int experience = 150
- boolEnemyImage : Image
+ EnemyBeastTwoPosition position() Constructor
+ attack : void(void)
+ void setEnemyPropertyBeta(EnemyProperty[] bevPropertyA[] assignment);
+ boolean isBoss();
+ void addPropertyBeta();
+ void setEnemyBeastList(EnemyProperty enemies)
EnemyProperty(abstract class)
+ void attack(LoopIterator&world, List<Ally> deltaAllies, List<EnemyProperty> deltaEnemies, List<EnemyProperty> transferZombies, boolean inBattle, NextPhase nextPhase);
+ boolean isBoss();
+ void setEnemyPropertyBeta();
BaseEnemy
- int: String
- int: int
- int: int
- int: int
- critical : boolean
- int: boolean
- fightRadius : int
- supportRadius : int
- int: int
- speed : int
+ Enemy(): Constructor
+ getID(): String
+ setID(): void
+ getRow(): int
+ getDamage(): int
+ setDamage(int): void
+ getSupportRadius(): int
+ getIsCritical(): boolean
+ setIsCritical(): void
+ getSize(): int
+ getSpeed(): int
+ setSpeed(int): void
+ setXSitting (x): void
+ setYSitting (y): void
+ setExp(int exp): void
+ setFightRadius(int radius): void
+ setSupportRadius(int radius): void
+ setIsCritical(boolean flag): void
+ setIsWeak(boolean flag): void
+ setSize(int grid): void
+ setIsBoss(boolean flag): void
+ setIsBoss(boolean flag)
+ setIsBoss(): void
+ attack_ally(Ally ally): void
+ attack_hero(Character c): void
+ attack(): void
+ getDistance(int x, int y): int
+ getBeftPropertyBeta(): void
Zombie
- String type "Zombie"
- int hp = 50
- int bugsoHP = 0
- int gold = 0
- int speed = 2
- int damage = 0
- boolean weak: false
- int bp: 500
- int exp = 1
+ Zombie(): constructor
+ void attack_ally(Ally ally)
+ void attack(LoopIterator&world, List<Ally> deltaAllies, List<EnemyProperty> deltaEnemies, List<EnemyProperty> transferZombies, boolEnemyProperty[] bevPropertyA[] assignment);
+ Imageview inLoseEnemy()
+ void setBeft()
+ void addPropertyBeta();
Doggie
- String type = "Doggie"
- int hp(int): 0
- int bugsoHP = 0
- int gold = 0
- int speed = 2
- int damage = 0
- boolean weak: false
- int bp: 500
- int exp = 0
- int dogsPrice
+ DoggieEnemyPosition position() Constructor
+ attack : void(void)
+ void setEnemyPropertyBeta()
+ void setPropertyBeta(): void
+ void setNextCharacter(Character c): void
+ isBoss(): boolean
Vampire
- String type "Vampire"
- int hp = 50
- int bugsoHP = 0
- int gold = 50
- int speed = 0
- int damage = 0
- boolean weak = false
- int bp = 500
- int exp = 50
- int vampireHP
+ Vampire(): Constructor
+ void attack_ally(Ally ally)
+ void attack_character(Character c)
+ void setEnemyPropertyBeta()
+ void setCurrentBeta()
+ void setBeftPropertyBeta()
+ void attack(LoopIterator&world, List<Ally> deltaAllies, List<EnemyProperty> deltaEnemies);
+ boolean isBoss();
+ void setBeftPropertyBeta()
«interface»
AllyState(State pattern)
Ally
- String type
- int hp
- int damage
- int range
- int speed
- String originalType
+ getType(): String
+ getID(): int
+ setID(int): void
+ int getRowd
+ void setRowd(int)
+ String getOriginalType
+ void setOriginalType
+ void attack()
+ getDamage(): int
WeakEntity
- PathPosition position
+ WeakEntity(): Constructor
+ void setPosition(int)
+ void moveCycle()
+ int: SimpleIntegerProperty
+ int: SimpleIntegerProperty
+ getID(): int
+ setID(int): void
+ getPathPosition(): PathPosition
Item
- ItemType type
- String name
+ Item(): Constructor
+ ItemType(): Constructor
+ getID(): int
ItemType
- index: int
+ ItemType(): Constructor
+ getIndex(): int
StaticEntity
- ImageProperty x
- ImageProperty x
+ StaticEntity(): Constructor
+ int: ImageProperty
+ (): ImageProperty
+ getX(): int
+ getY(): int
+ setX(int x)
+ void setY(int y)
Entity
- BooleanProperty shouldExist;
+ void destroy()
+ BooleanProperty shouldExist()
+ void setShouldExist(int)
+ abstract ImageProperty y()
+ abstract int getID()
+ abstract int getY()
0...n
1
Building
- List<BaseEnemy> enemies
- boolean characterStepOn
- int pathCycle
+ Building(): Constructor
+ getPathCircle() :int
+ void addEnemy(BaseEnemy)
+ getCharacterStepon(): boolean
+ int getID()
+ getY() :int
+ void setPathCircle(int)
+ void setCharacterStepOn()
+ void removeEnemy(Entity deadOn)
+ void characterStepOn(Ally)
+ void characterLeave()
+ void
addEnemeesWithin(LoopIterator&world)
+ void setPropertyBeta(BuildingPropertyA)
PathPosition
+ getDistance(int x, int y): double
BuildingProperty(abstract class)
+ void spawnEnemy(LoopIterator&world, List<EnemyProperty> spawningEnemies);
+ void characterStepOn(LoopIterator&world);
+ void addBuilding(LoopIterator&world, List<BuildingProperty> buffenemies)
+ Imageview inLoseBuilding()
Trap
+ void debuffAllEnemy stepOn()
Barracks
+ Barracks(): Constructor
+ protectedAlly(LoopIterator&world): Ally
Zombie pit
+ ZombiePit(): Constructor
+ void spawnZombie(LoopIterator&world)
+ int: Zombie
+ checkPathCircle(LoopIterator&world)
boolean
Tower
- int shootRadius: 5
- int damage: 5
+ Tower(): Constructor
+ void shootEnemy(LoopIterator&world)
+ int decreaseHp()
+ void attack(LoopIterator&world)
+ getNumberFoes(): int
Hero castle
- boolean offerWindow
+ HeroCastle(): Constructor
+ void addHero(Hero hero)
+ checkPathCircle(): boolean
+ getOfferWindow(): boolean
+ void setOfferWindow()
+ void closeOfferWindow()
Village
+ int hpGain
+ Village(): Constructor
+ getHpGain(): int
+ void increaseHp(Character c)
VampireCastleBuilding
+ VampireCastleBuilding(): Constructor
+ void spawnVampire(LoopIterator&world)
Vampire
+ checkPathCircle(LoopIterator&world)
boolean
Campfire
- int campRadius: 3
+ void doubleDamgeeCharacter(c)
+ CampFire(): Constructor
+ getcampRadius(): int
TowerCard
+ TowerCard: Constructor
VillageCard
+ VillageCard(): Constructor
ZombiePitCard
+ ZombiePitCard Constructor
CampfireCard
+ field: type
+ CampfireCard(): Constructor
BarracksCard
+ BarracksCard(): Constructor
CardProperty
+ CardProperty(): Constructor
+ void(LoopIteratorWorld&Controller, GridPane terrain, GridPane roadmap): int
VampireCastleCard
+ VampireCastleCard(): Constructor
TrapCard
+ TrapCard: Constructor
Card
- String type;
+ Card: Constructor
+String getType()
Armour
- int defense: 200
- int price: 2000
+ Arrmour(): Constructor
+ getDefense(): int
+ getPrice(): int
+ setDefense(int): void
+ void(LoopIteratorWorld&world) ;
+ void setPropertyA(int v);
+ int getDefense();
+ Imageview
+ setBuyPurchased(): boolean
The One Ring
TheOneRing(): Constructor
+ void useOnceBlade
+ void characterStepOn
+ int getPrice()
+ ImageView inLoseItem()
+ boolean canBePurchased()
Anduril
- final int damage = 250
+ Image anduriImage
+ void useOnceBlade
+ void characterStepOn
+ int getPrice()
+ ImageView inLoseItem()
+ boolean canBePurchased()
TreeStump
- final int defense = 20;
- Image treeStumpImage;
+ int getDefense()
+ int getPrice()
+ void useOnceBlade
+ ImageView inLoseItem()
+ void addPropertyBeta(LoopIterator&world) ; List<ItemProperty> bifferenceGold ; List<ItemProperty> bifferenceHealthPotion ;
+ boolean canBePurchased()
Basic Item
- ItemType type
+ BaseItem(): Constructor
Rare Item
- ItemType type
RareItem(): Constructor
Sword
- int damage: 200
- int price: 1000
+ Sword(): Constructor
+ getDamage(): int
+ getPrice(): int
+ void setDamage(int)
+ void(LoopIteratorWorld&world) ;
+ void setPropertyA(int v);
+ int getDamage();
+ Imageview
+ setBuyPurchased(): boolean
Helmet
- int defense: 150
- int price: 1500
+ helmet(): Constructor
+ getDefense(): int
+ getPrice(): int
+ void setDamageEnemyProperty e, Character c) ;void(void);
+ characterStepOn(LoopIteratorWorld) ;
+ void setPrice();
+ ImageView inLoseItem() ; Imageview
+ canBePurchased(): boolean
Shoe
- int damage: 150
- int price: 1500
+ Shoe(): Constructor
+ getDefense(): int
+ getPrice(): int
+ void setDamageEnemyProperty e, Character c) ;void(void);
+ characterStepOn(LoopIteratorWorld) ;
+ void setPrice();
+ ImageView inLoseItem() ; Imageview
+ canBePurchased(): boolean
Staff
- int damage: 100
- int price: 2000
+ Staff(): Constructor
+ getDefense(): int
+ getPrice(): int
+ void transferBaseEnemy e, Character c) ;void(void);
+ void setDamageEnemyProperty e, Character c) ;void(void);
+ characterStepOn(LoopIteratorWorld) ;
+ void setPrice();
+ ImageView inLoseItem() ; Imageview
+ canBePurchased(): boolean
ItemProperty
+ void useDoingBeftEnemyProperty e, Character c) ;void(void);
+ characterStepOn(LoopIteratorWorld) ;List<ItemProperty> bifferenceGold ; List<ItemProperty> bifferenceHealthPotion ;
+ int getPrice();
+ void setPrice();
+ void canBePurchased();
+ boolean canBePurchased();
DoggieCoin
- price : int
+ DoggieCoinOnsprimeagePropery ;
+ void(LoopIteratorWorld&world) ;
+ void useDoingBeftEnemyProperty e, Character c) ;void(void);
+ void(LoopIteratorWorld&world) ;List<ItemProperty> cr List<ItemproperyA> c) ; void
+ int getPrice();
+ setPrice(int value): void
+ imageBeftsc() ; ImageValue
+ canBePurchased(): boolean
Health potion
- int health: 200
- int price: 2000
+ HealthPotion(): Constructor
+ getHealth(): int
+ void transferBaseEnemyProperty e, Character c) ;void(void);
+ characterStepOn(LoopIteratorWorld) ;
+ void setPrice();
+ ImageView inLoseItem() ; Imageview
+ canBePurchased(): boolean
Gold
- int value: 200
+ Gold(): Constructor
+ getValue(): int
+ void useDoingBeftEnemyProperty e, Character c) ;void(void);
+ characterStepOn(LoopIteratorWorld) ;
+ void setPrice();
+ ImageView inLoseItem() ; Imageview
+ canBePurchased(): boolean
Shield
- int defense: 200
- int price: 2000
+ Shield(): Constructor
+ getDefense(): int
+ getPrice(): int
+ void useDoingBeftEnemyProperty e, Character c) ;void(void);
+ characterStepOn(LoopIteratorWorld) ;
+ void setPrice();
+ ImageView inLoseItem() ; Imageview
+ canBePurchased(): boolean